

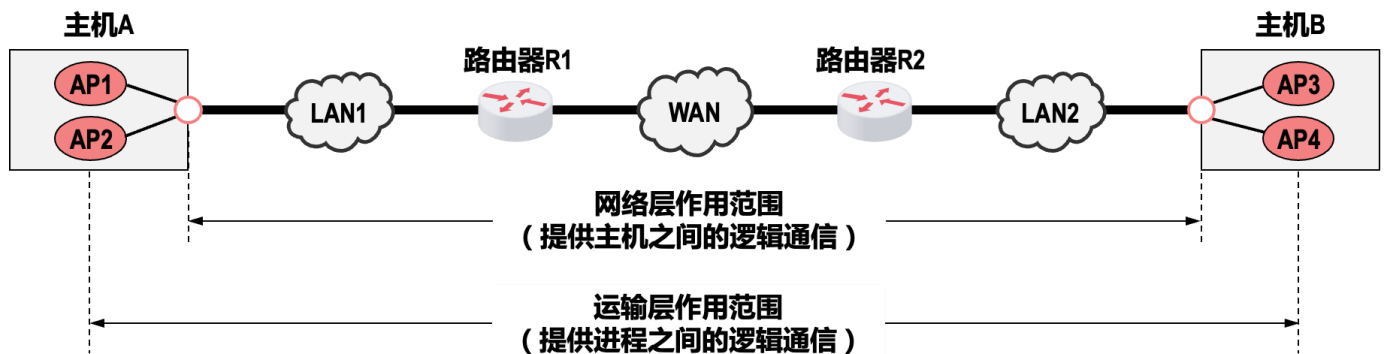
6. 传输层

1. 传输层概述

1.1. 传输层的作用

计算机网络体系结构中的物理层、数据链路层和网络层，它们共同解决了将主机通过异构网络互联起来所面临的问题，实现了**主机到主机的通信**。

如何**为运行在不同主机上的应用进程提供直接的逻辑通信服务**，就是传输层的主要任务，传输层协议又称为端到端协议。



传输层提供了“端口”的概念，用来区分不同应用进程的标识符。传输层向应用层实体屏蔽了下面网络核心的细节（例如网络拓扑、所采用的路由选择协议等），它使应用进程看见的就好像是在两个传输层实体之间有一条端到端的逻辑通信信道。

根据应用需求的不同，因特网的传输层为应用层提供了两种不同的传输层协议，即面向连接的TCP和无连接的UDP，这两种协议就是本章要讨论的主要内容。

1.2. TCP/IP传输层中的两个重要协议

1.2.1. TCP

- **传输控制协议**（Transmission Control Protocol, TCP）为其上层提供的是**面向连接的可靠**的数据传输服务。
- 使用TCP通信的双方，在传送数据之前**必须首先建立TCP连接**（逻辑连接，而非物理连接）。数据传输结束后**必须要释放TCP连接**。
- TCP为了实现可靠传输，就必须使用很多措施，例如**TCP连接管理、确认机制、超时重传、流量控制以及拥塞控制**等。

- TCP的实现复杂，TCP报文段的**首部比较大，占用处理机资源比较多**。

1.2.2. UDP

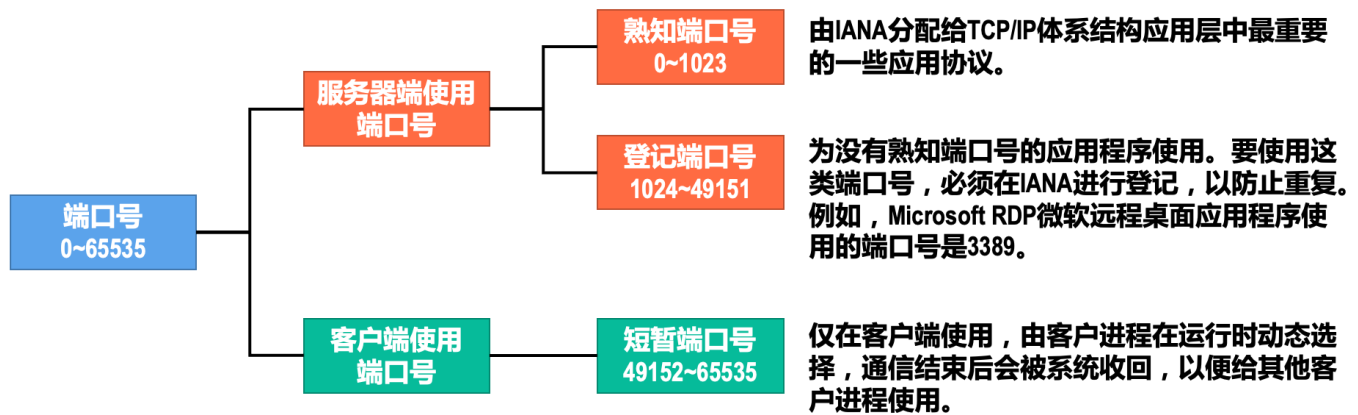
- **用户数据报协议**（User Datagram Protocol，UDP）为其上层提供的是**无连接的不可靠**的数据传输服务。
- 使用UDP通信的双方，在传送数据之前**不需要建立连接**。
- UDP不需要实现可靠传输，因此**不需要使用实现可靠传输的各种机制**。
- UDP的实现简单，UDP用户数据报的**首部比较小**。

1.2.3. 因特网中典型应用使用的运输层协议

因特网应用	TCP/IP应用层协议	TCP/IP运输层协议
域名解析	域名系统DNS	UDP
文件传送	简单文件传送协议TFTP	UDP
路由选择	路由信息协议RIP	UDP
网络参数配置	动态主机配置协议DHCP	UDP
网络管理	简单网络管理协议SNMP	UDP
远程文件服务器	网络文件系统NFS	UDP
IP电话	专用协议	UDP
流媒体通信	专用协议	UDP
IP多播	网际组管理协议IGMP	UDP
电子邮件	简单邮件传送协议SMTP	TCP
远程终端接入	电传机网络TELNET	TCP
万维网	超文本传送协议HTTP	TCP
文件传送	文件传送协议FTP	TCP

1.3. 传输层端口号

- 运行在计算机上的进程是使用进程标识符（Process Identification，PID）来标识的。
 - 然而，因特网上的计算机并不是使用统一的操作系统，而**不同操作系统**（Windows、Linux、MacOS）又**使用不同格式的进程标识符**。
 - 为了使运行不同操作系统的计算机的应用进程之间能够基于网络进行通信，就必须**使用统一的方法对TCP/IP体系的应用进程进行标识**。
- TCP/IP体系结构的运输层使用**端口号**来标识和区分应用层的不同应用进程。端口号的**长度为16比特**，取值范围是0~65535。
- 端口号的分类



FTP	SMTP	DNS	DHCP	HTTP	BGP	HTTPS	RIP
21/20	25	53	67/68	80	179	443	520

TCP和UDP的端口号之间是没有关系的，他们是独立的。

1.4. UDP和TCP的对比

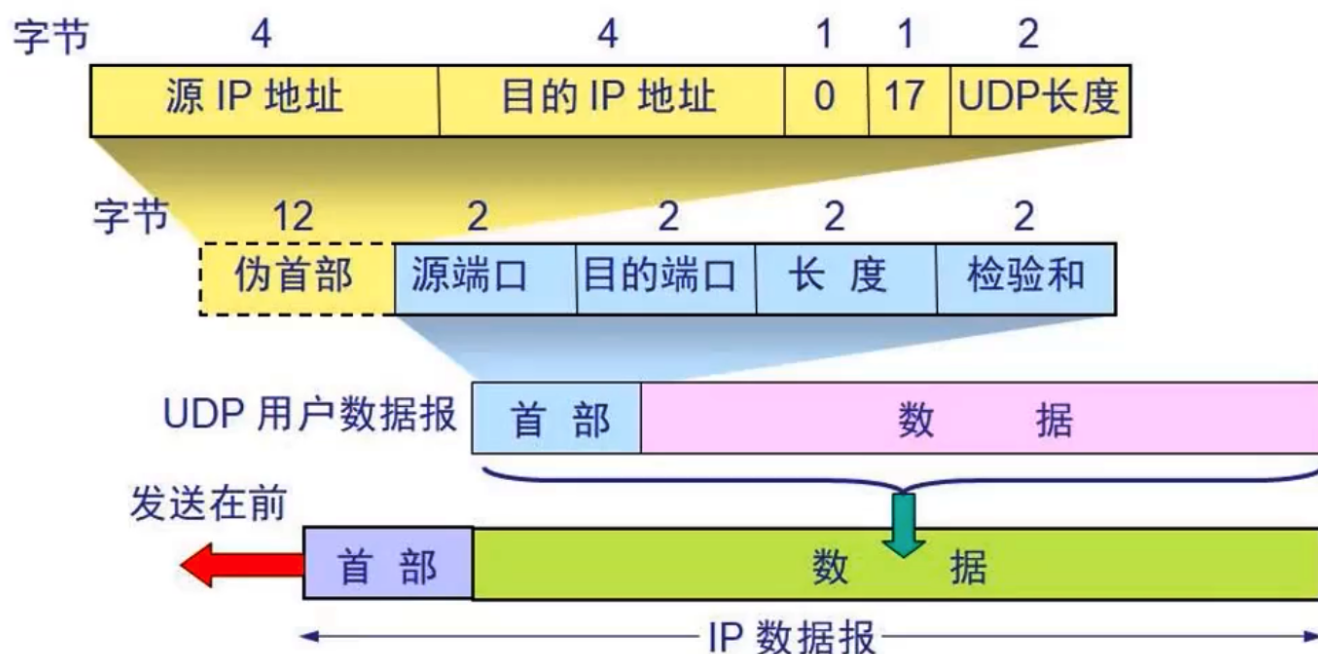
	TCP	UDP
连接性	面向连接	无连接
可靠性	可靠传输，不丢包	不可靠传输，尽最大努力交付，可能丢包
首部占用空间	大	小
传输速率	慢	快
资源消耗	大	小
应用场景	浏览器、文件传输、邮件发送	音视频通话、直播
应用层协议	HTTP、HTTPS、FTP、SMTP、DNS	DNS

2. UDP报文段格式

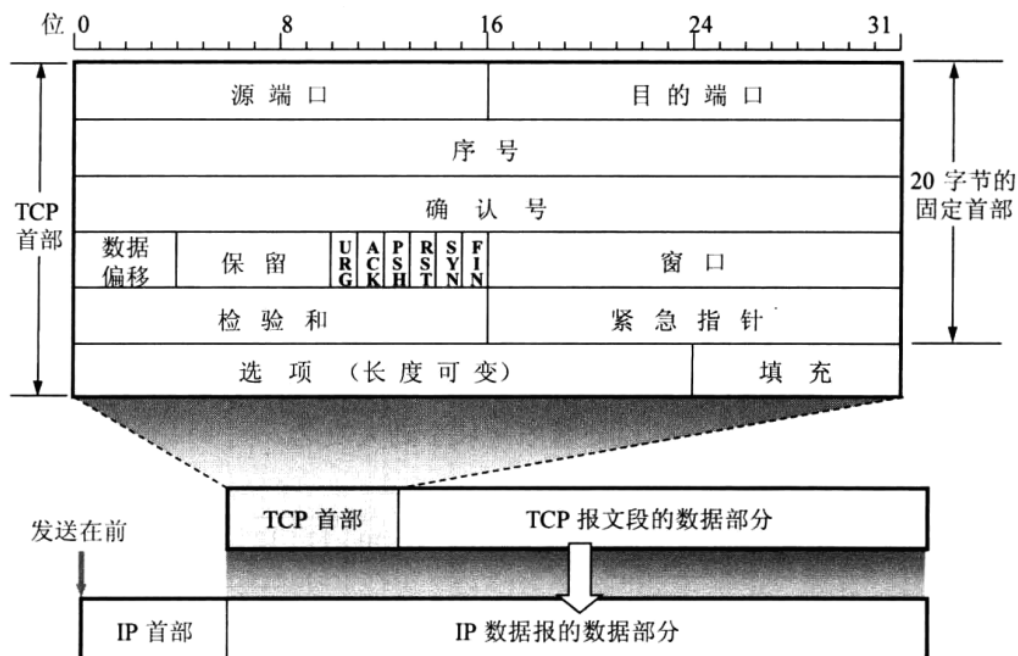
- UDP是无连接的，减少了建立和释放连接的开销
- UDP尽最大能力交付，不保证可靠交付
 - 不需要维护复杂的参数，首部只有8个字节（TCP首部至少20字节）



- UDP长度
 - 占16位，首部的长度+数据长度
- 校验和的计算内容：伪首部 + 首部 + 数据
 - 伪首部：仅在计算校验和时起作用，并不会传递给网络层



3. TCP报文段格式



- 序号

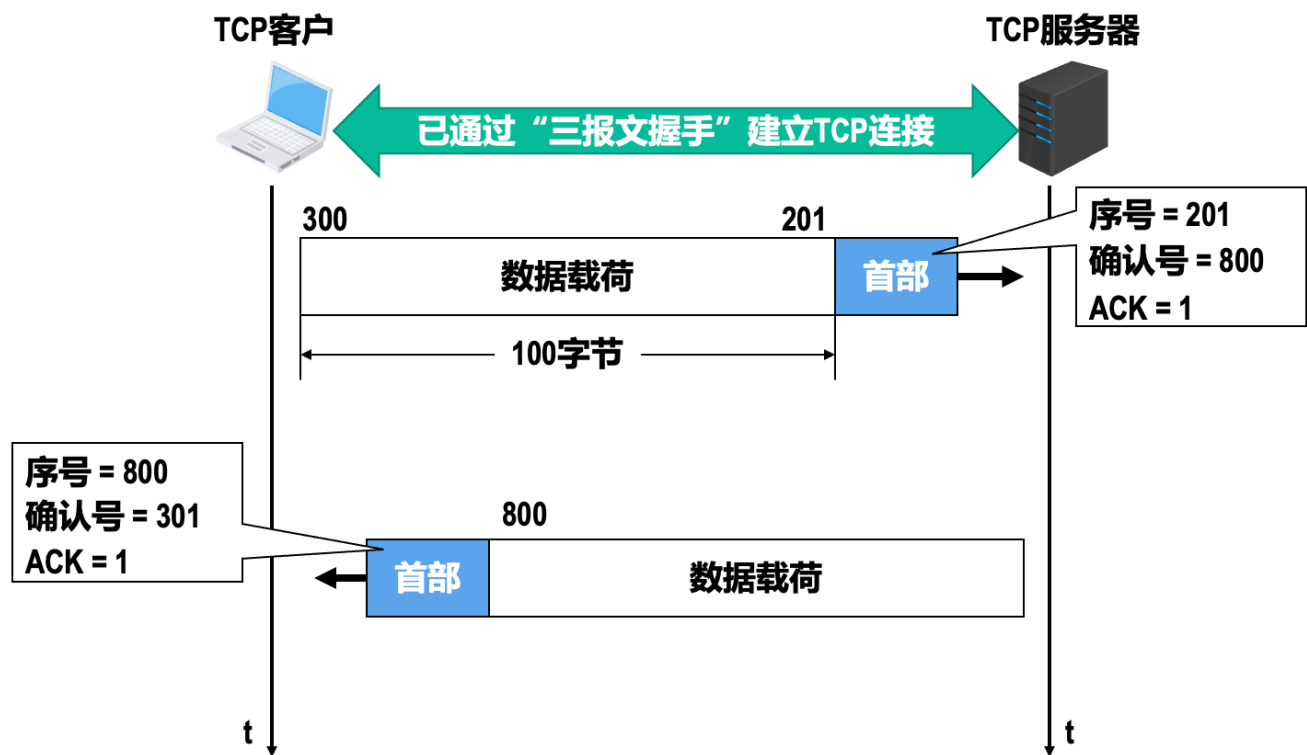
占32比特，取值范围 $0 \sim 2^{32} - 1$ 。当序号增加到最后一个时，下一个序号又回到0。用来指出本TCP报文段数据载荷的第一个字节的序号。

- 确认号

占32比特，取值范围 $0 \sim 2^{32} - 1$ 。当确认号增加到最后一个时，下一个确认号又回到0。用来指出期望收到对方下一个TCP报文段的数据载荷的第一个字节的序号，同时也是对之前收到的所有数据的确认。

- 确认标志位ACK

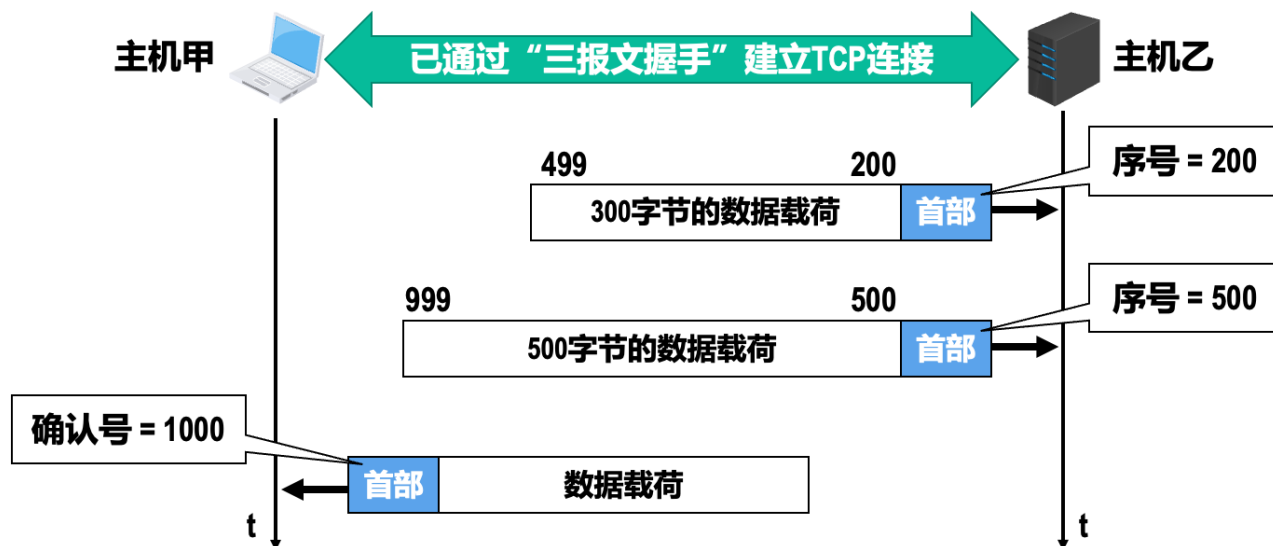
只有当ACK取值为1时，确认号字段才有效。ACK取值为0时，确认号字段无效。TCP规定：在TCP连接建立后，所有传送的TCP报文段都必须把ACK置1。



▼ 练习题

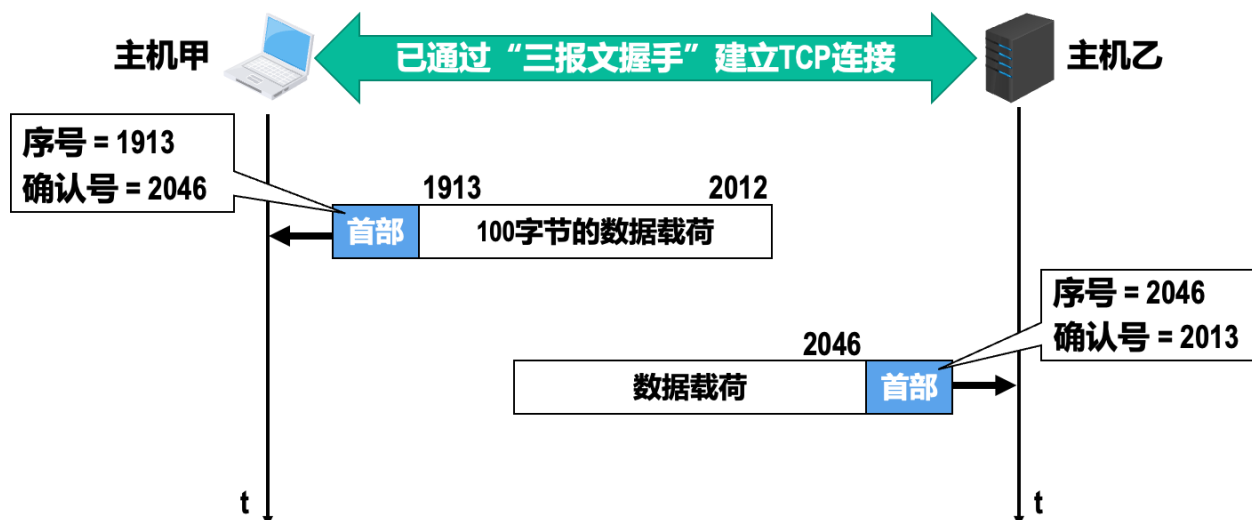
1. 【2009年 题38】主机甲与主机乙之间已建立一个TCP连接，主机甲向主机乙发送了两个连续的TCP段，分别包含300字节和500字节的有效载荷，第一个段的序列号为200，主机乙正确接收到两个段后，发送给主机甲的确认序列号是（ ）。

A、500 B、700 C、800 D、1000



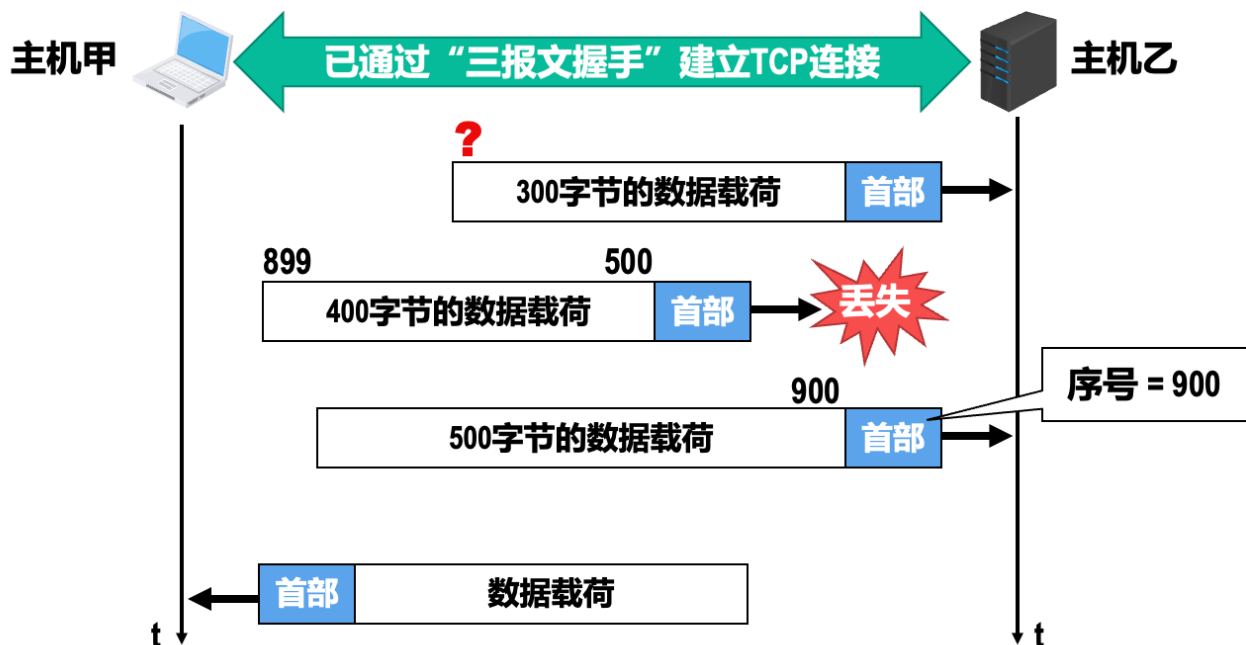
2. 【2013年 题39】主机甲与主机乙之间已建立一个TCP连接，双方持续有数据传输，且数据无差错与丢失。若甲收到1个来自乙的TCP段，该段的序号是1913、确认序号为2046、有效载荷为100字节，则甲立即发送给乙的TCP段的序号和确认序号分别是（ ）。

A、2046、2012 B、2046、2013 C、2047、2012 D、2047、2013



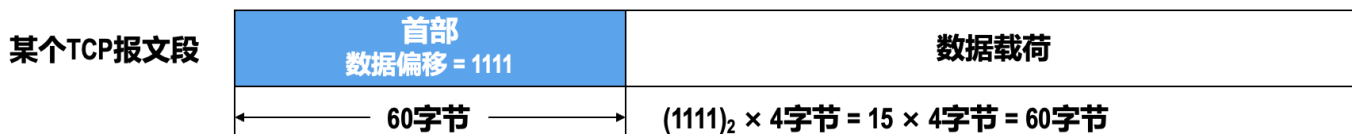
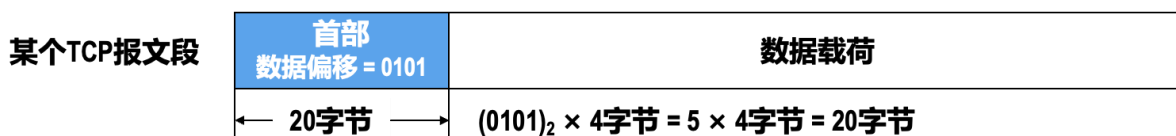
3. 【2011年 题40】主机甲与主机乙之间已建立一个TCP连接，主机甲向主机乙发送了3个连续的TCP段，分别包含300字节、400字节和500字节的有效载荷，第3个段的序号为900，若主机乙仅正确接收到第1 和第3个段，则主机乙发送给主机甲的确认序号是（ ）。

A、300 B、500 C、1200 D、1400



- 数据偏移

- 占4比特，该字段的取值以4字节为单位。
- 指出TCP报文段的数据载荷部分的起始处距离TCP报文段的起始处有多远，这实际上指出了TCP报文段的首部长度。

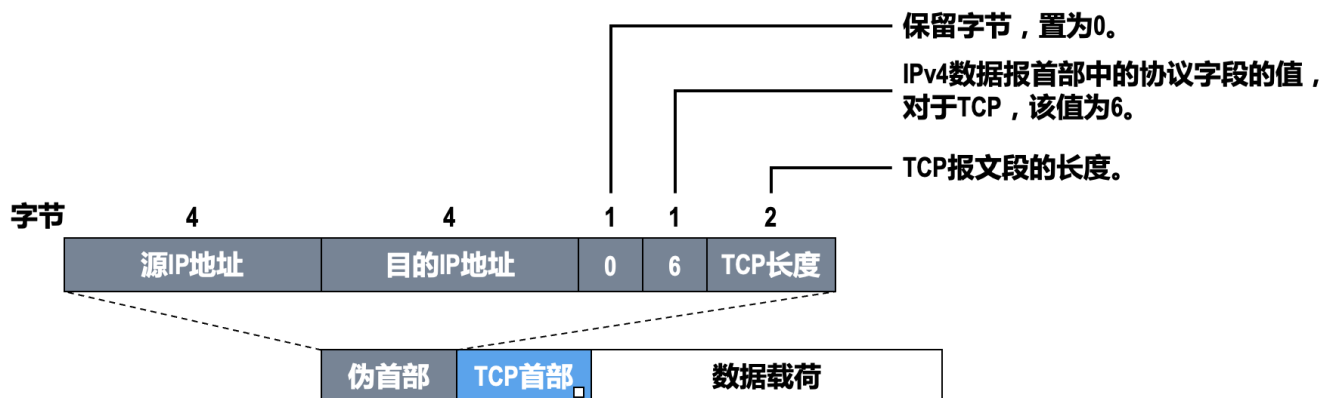


- 窗口

- 占16比特，该字段的取值以字节为单位。
- 指出发送本报文段的一方的接收窗口的大小，即接收缓存的可用空间大小，这用来表征接收方的接收能力。
- 在计算机网络中，经常用接收方的接收能力的大小来控制发送方的数据发送量，这就是所谓的流量控制。

- 校验和

- 占16比特
- 用来检查整个TCP报文段在传输过程中是否出现了误码。



○ 计算方法

- ①. 将TCP首部中检验和字段的值置为0。
- ②. 将伪首部、TCP首部以及数据载荷这三部分划分成若干个2字节的字。若伪首部、TCP首部以及数据载荷这三部分的总长度不是偶数个字节，则在最后添加1个“全0”字节。
- ③. 对划分出的全部2字节的字进行反码算数运算求和，并将求和结果取反码。
- ④. 将步骤③得到的结果写入TCP首部中的检验和字段。

● 标志位

○ 同步标志位SYN

- SYN为1的TCP报文段要么是一个连接请求报文段，要么是一个连接响应报文段。

○ 终止标志位FIN

- 用于TCP“四报文挥手”释放连接。
- 当FIN=1时，表明此TCP报文段的发送方已经将全部数据发送完毕，现在要求释放TCP连接。

○ 复位标志位RST

- 用于复位TCP连接。
- 当RST=1时，表明TCP连接中出现严重差错，必须释放连接，然后再重新建立连接。
- RST置1还用来拒绝一个非法的TCP报文段或拒绝打开一个TCP连接。

○ 推送标志位PSH

- 发送方TCP把PSH置1，并立即创建一个TCP报文段发送出去，而不需要积累到足够多的数据再发送。
- 接收方TCP收到PSH为1的TCP报文段，就尽快地交付给应用进程，而不再等到接收到足够多的数据才向上交付。

● 选项（长度可变，最大40字节）

- 最大报文段长度MSS选项：指出TCP报文段数据载荷部分的最大长度，而不是整个TCP报文段

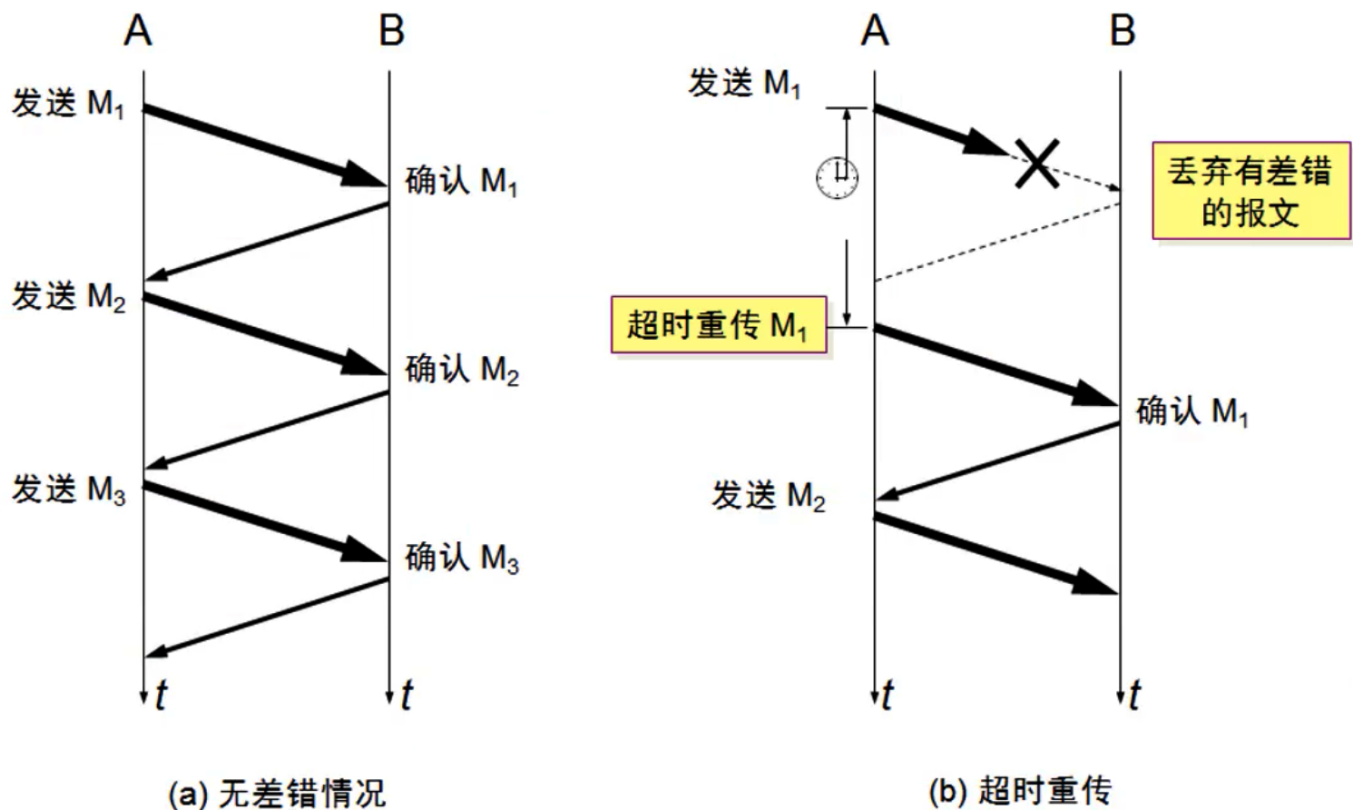
的长度。

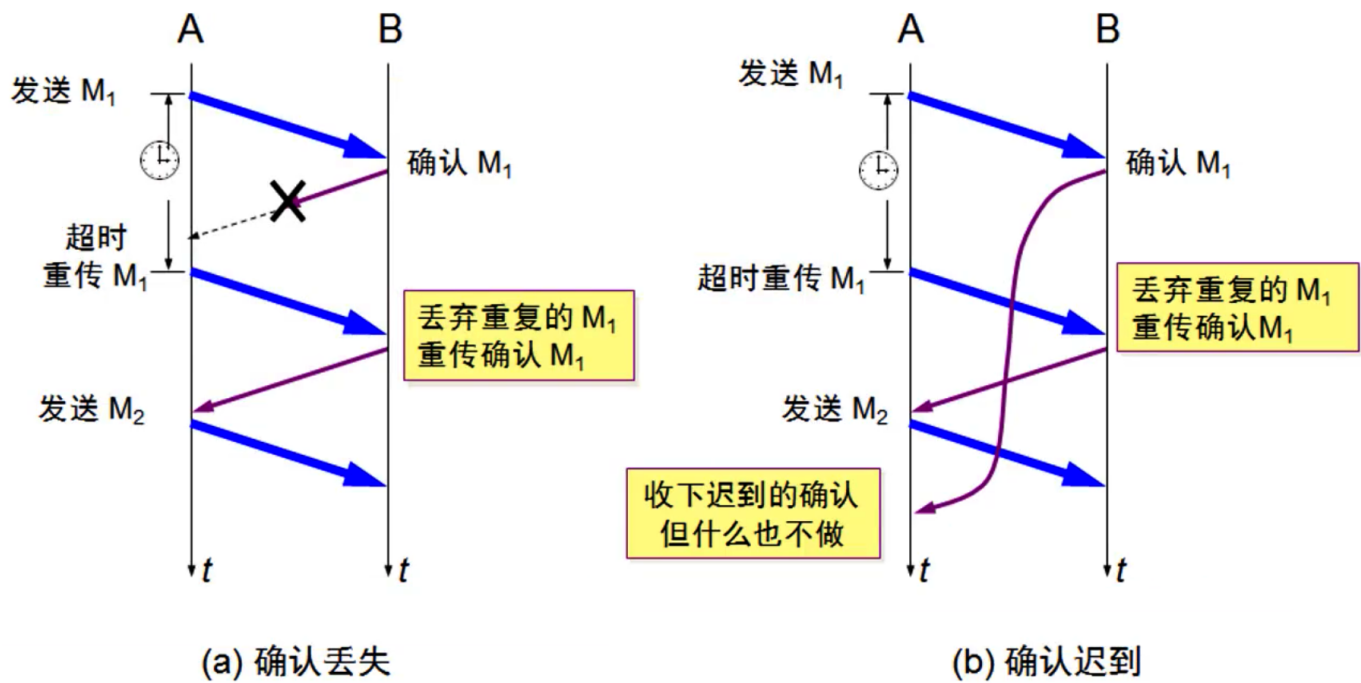
- 窗口扩大选项：用来扩大窗口，提高吞吐率。
- 时间戳选项：
 - 用于计算往返时间RTT
 - 用于处理序号超范围的情况，又称为防止序号绕回PAWS。
- 选择确认选项：用来实现选择确认功能。

4. TCP可靠传输

4.1. 停止等待ARQ协议

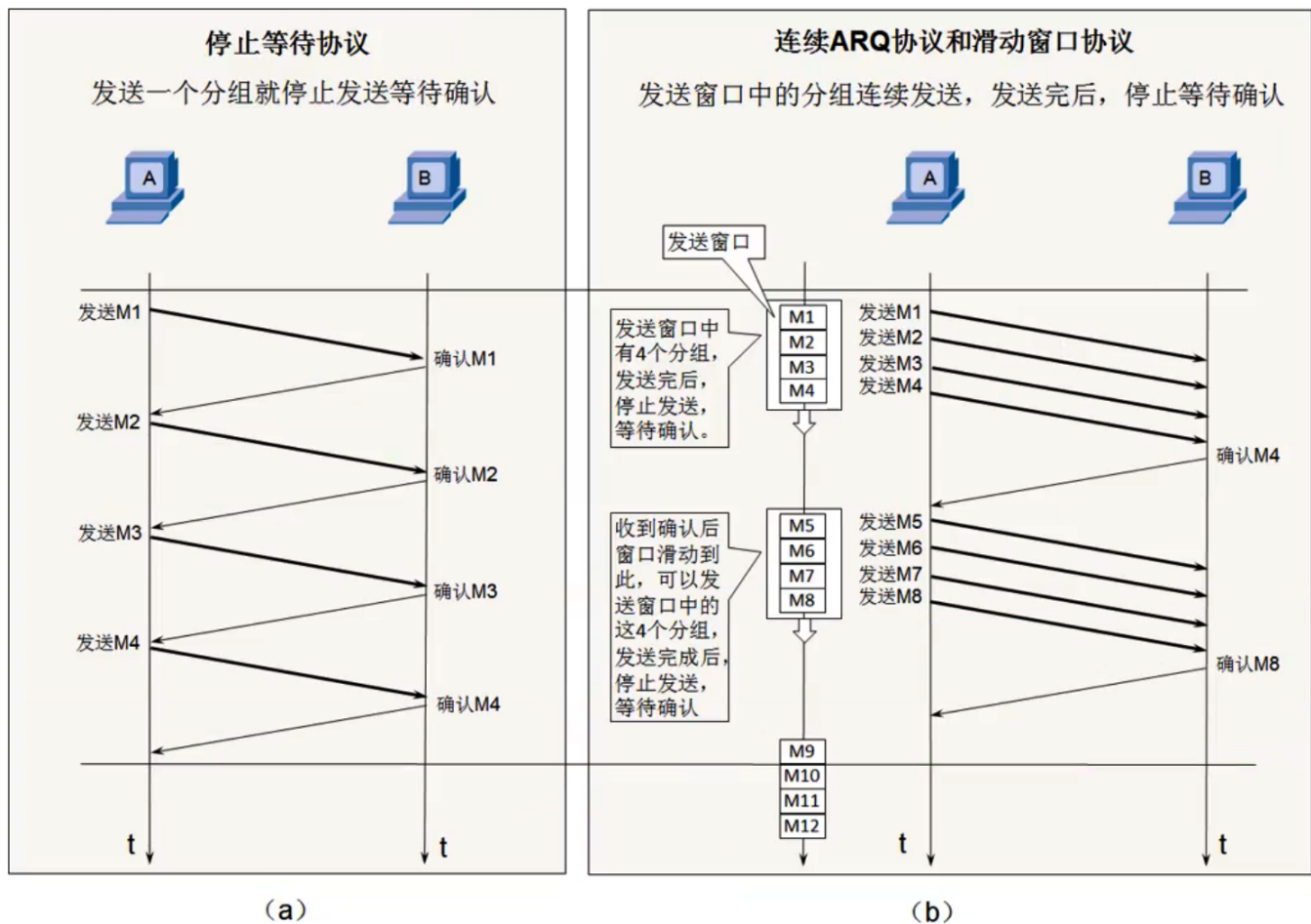
- ARQ (Automatic Repeat-reQuest) 自动重传请求



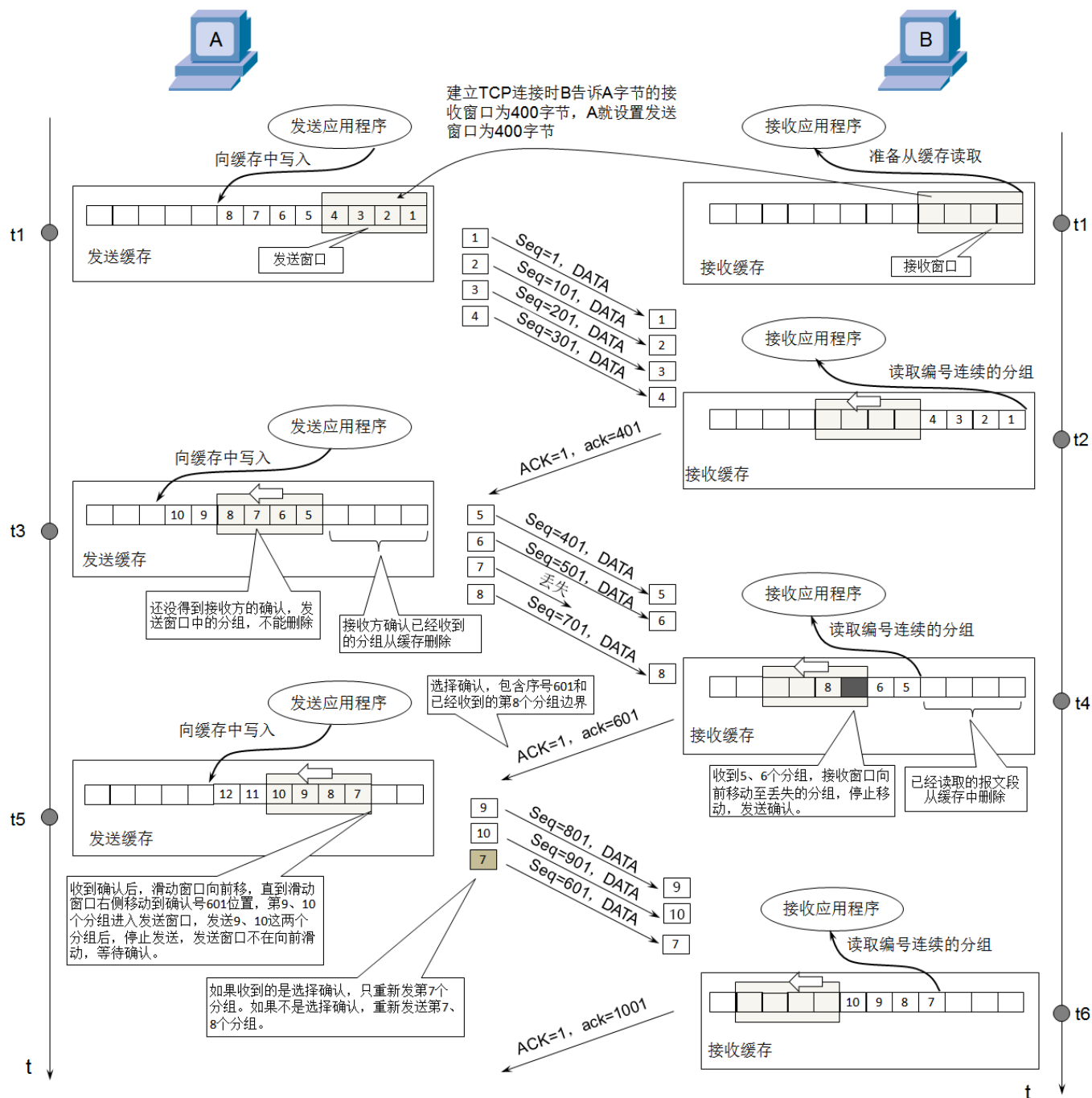


- 若有个包重传了N次还是失败，会一直持续重传到成功为止吗？
 - 取决于系统的设置，重传5次还未成功就会发送RST报文，断开连接

4.2. 连续ARQ协议 + 滑动窗口协议



- 如果接收窗口最多能接4个包，但发送方只发了2个包
 - 接收方等待一段时间后，没有收到第3个包，就会返回确认收到2个包给发送方
- 每个包多大那，每个确认编号如何编址
- 序号和确认号



4.3. 选择性确认 (SACK)

在TCP通信过程中，如果发送序列中间某个数据包丢失（比如1、2、3、4、5中的3丢失了），TCP会通过重传最后确认的分组后续的分组（最后确认的是2，会重传3、4、5），这样原先已经正确传输的分组也可能重复发送（比如4、5），降低了TCP性能。

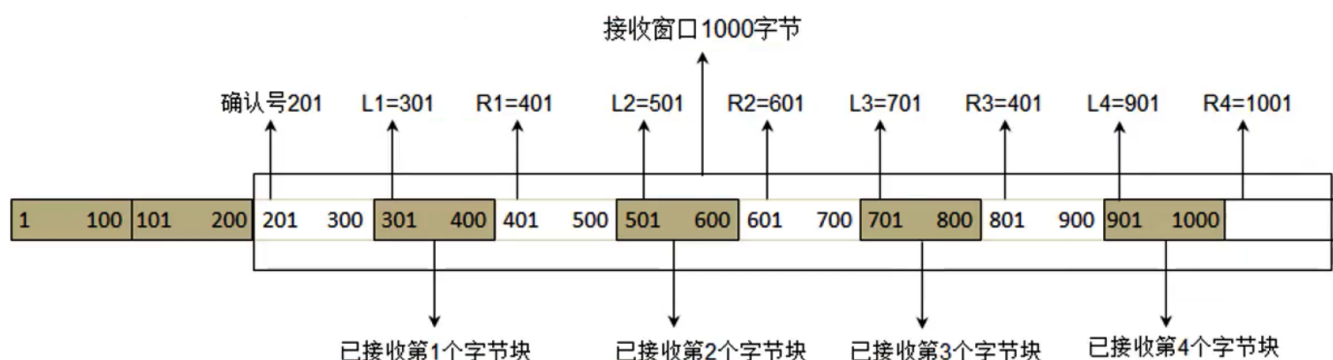
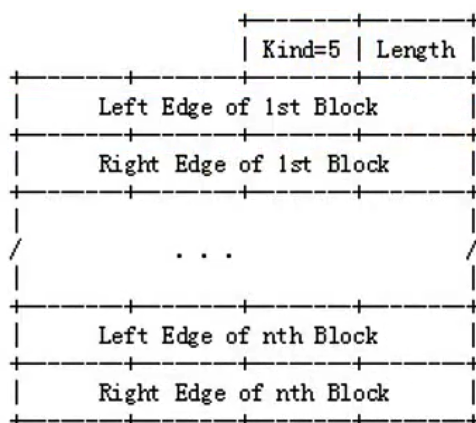
- 发展出了SACK (Selective acknowledgment, 选择性确认) 技术
 - 告诉发送方哪些数据丢失，哪些数据已经提前收到
 - 使TCP只重新发送丢失的包（比如3），不用发送后续所有的分组（比如4、5）

- SACK信息会放在TCP首部的选项部分
 - Kind: 占1字节。值为5代表这是SACK选项
 - Length: 占1字节。表明SACK选项一共占用多少字节
 - Left Edge: 占4字节，左边界
 - Right Edge: 占4字节，右边界

TCP SACK Option:

Kind: 5

Length: Variable



- 一对边界信息需要占用8字节，由于TCP首部的选项部分最多40字节
 - SACK选项最多携带4组边界信息
 - SACK选项的最大占用字节数 = $4 * 8 + 2 = 34$

788 4.494645	192.168.3.3	113.96.140.220	TCP	54 4357 → 80 [ACK] Seq=1 Ack=1 Win=262400 Len=0
789 4.497052	113.96.140.220	192.168.3.3	TCP	60 80 → 4358 [ACK] Seq=1 Ack=748 Win=11008 Len=0
795 4.509235	113.96.140.220	192.168.3.3	TCP	366 80 → 4358 [PSH, ACK] Seq=1 Ack=748 Win=11008 Len=312 [TCP segment of a reassembled PDU]
796 4.509235	113.96.140.220	192.168.3.3	TCP	1334 80 → 4358 [ACK] Seq=313 Ack=748 Win=11008 Len=1280 [TCP segment of a reassembled PDU]
797 4.509235	113.96.140.220	192.168.3.3	TCP	1334 80 → 4358 [ACK] Seq=1593 Ack=748 Win=11008 Len=1280 [TCP segment of a reassembled PDU]
798 4.509235	113.96.140.220	192.168.3.3	TCP	1334 80 → 4358 [ACK] Seq=2873 Ack=748 Win=11008 Len=1280 [TCP segment of a reassembled PDU]
799 4.509235	113.96.140.220	192.168.3.3	TCP	1334 80 → 4358 [ACK] Seq=4153 Ack=748 Win=11008 Len=1280 [TCP segment of a reassembled PDU]
800 4.509359	192.168.3.3	113.96.140.220	TCP	54 4358 → 80 [ACK] Seq=748 Ack=5433 Win=262400 Len=0
801 4.509385	113.96.140.220	192.168.3.3	TCP	1334 80 → 4358 [ACK] Seq=5433 Ack=748 Win=11008 Len=1280 [TCP segment of a reassembled PDU]
802 4.509385	113.96.140.220	192.168.3.3	TCP	1334 80 → 4358 [ACK] Seq=6713 Ack=748 Win=11008 Len=1280 [TCP segment of a reassembled PDU]
803 4.509385	113.96.140.220	192.168.3.3	TCP	1258 80 → 4358 [PSH, ACK] Seq=7993 Ack=748 Win=11008 Len=1204 [TCP segment of a reassembled PDU]
804 4.509385	113.96.140.220	192.168.3.3	HTTP	60 HTTP/1.1 200 OK (text/html)
805 4.509434	192.168.3.3	113.96.140.220	TCP	54 4358 → 80 [ACK] Seq=748 Ack=9202 Win=262400 Len=0
943 4.794324	192.168.3.3	113.96.140.220	HTTP	681 GET /favicon.ico HTTP/1.1

- 思考问题

- 为什么选择在传输层将数据分成多个段，而不是等到网络层分片传递给数据链路层

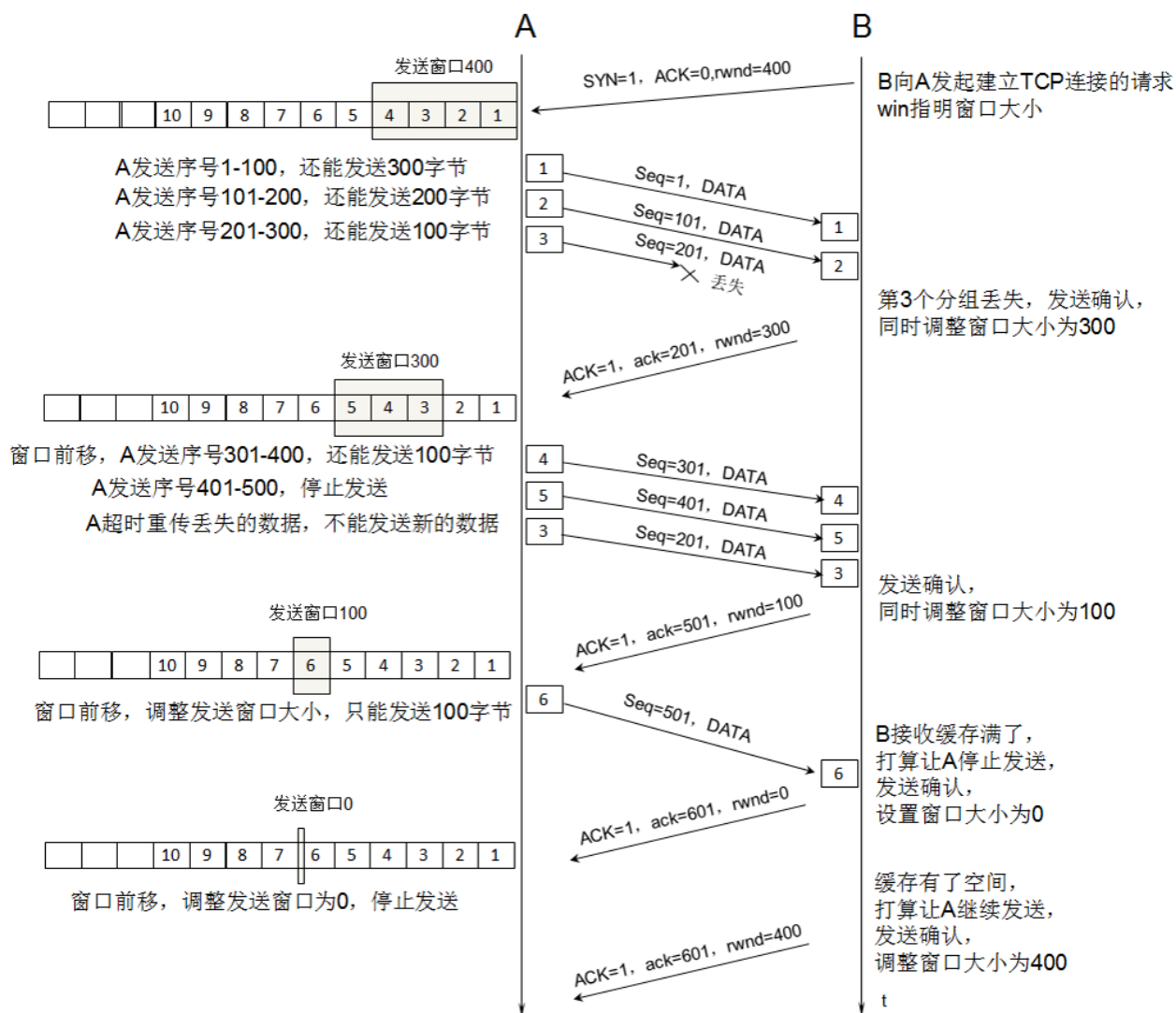
5. TCP流量控制

应用程序可能正忙于其他任务，并不一定能够立刻取走数据。客户端持续发送大量数据，接收端缓存会溢出，造成数据丢失。

TCP为应用程序提供了**流量控制**（Flow Control）机制，以**解决因发送方发送数据太快而导致接收方来不及接收，造成接收方的接收缓存溢出的问题**。

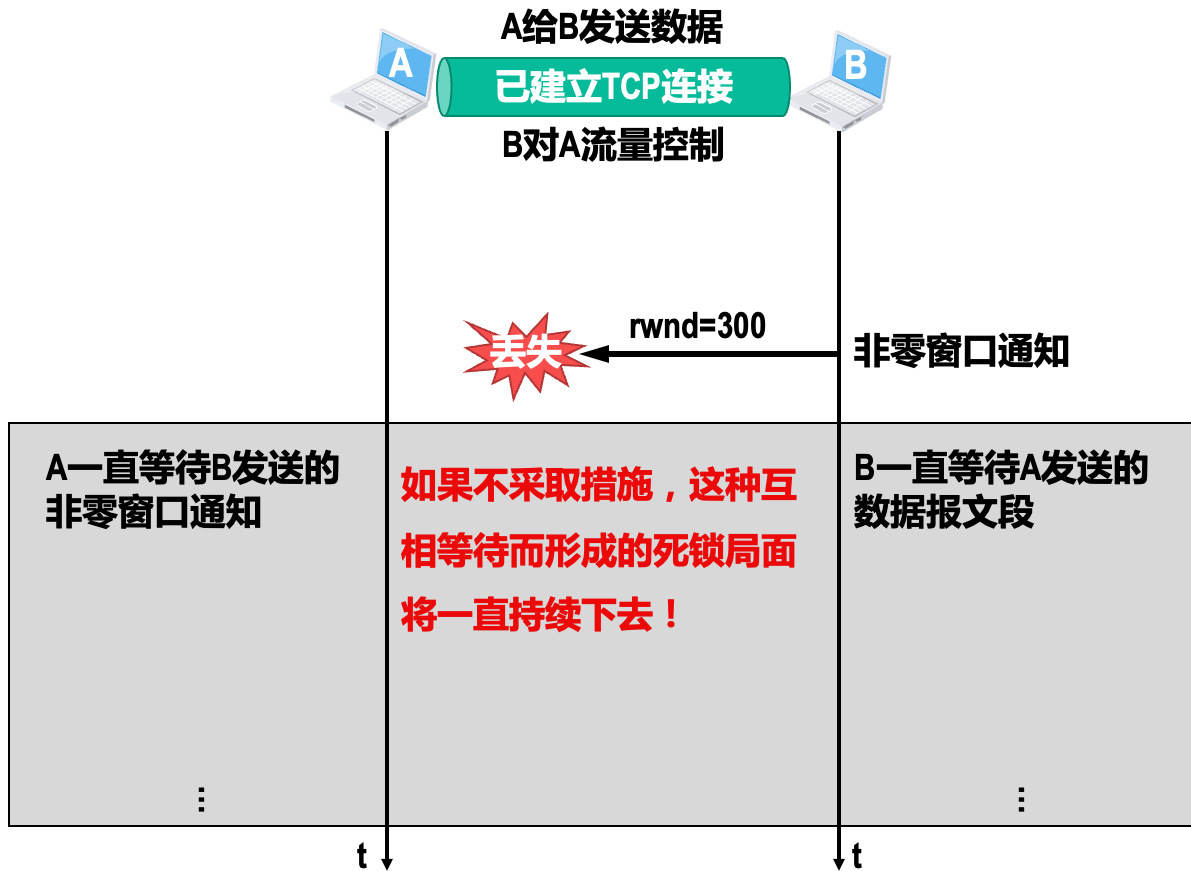
5.1. 基本情况

- 流量控制的基本方法：**接收方根据自己的接收能力（接收缓存的可用空间大小）控制发送方的发送速率**。
 - 通过确认报文中窗口字段来控制发送方的发送速率
 - 发送方的发送窗口大小不能超过接收方给出窗口大小
 - 当发送方收到接收窗口的大小为0时，发送方就会停止发送数据



5.2. 特殊情况

一开始，接收方给发送方发送了0窗口的报文段，后面，接收方又有了一些存储空间，给发送方发送的非0窗口的报文段丢失了，发送方的发送窗口一直为零，双方陷入僵局。



- 解决方案：
 - 当发送方收到0窗口通知时，这时发送方停止发送报文
 - 并且同时开启一个定时器，隔一段时间就发个测试报文去询问接收方最新的窗口大小
 - 如果接收的窗口大小还是为0，则发送方再次刷新启动定时器

▼ 小练习

1. 【2010年 题39】主机甲和主机乙之间建立了一个TCP连接，TCP最大段长度为1000字节。若主机甲的当前拥塞窗口为4000字节，在主机甲向主机乙连续发送两个最大段后，成功收到主机乙发送的第一个段的确认段，确认段中通告的接收窗口大小为2000字节，则此时主机甲还可以向主机乙发送的最大字节数是（ ）。

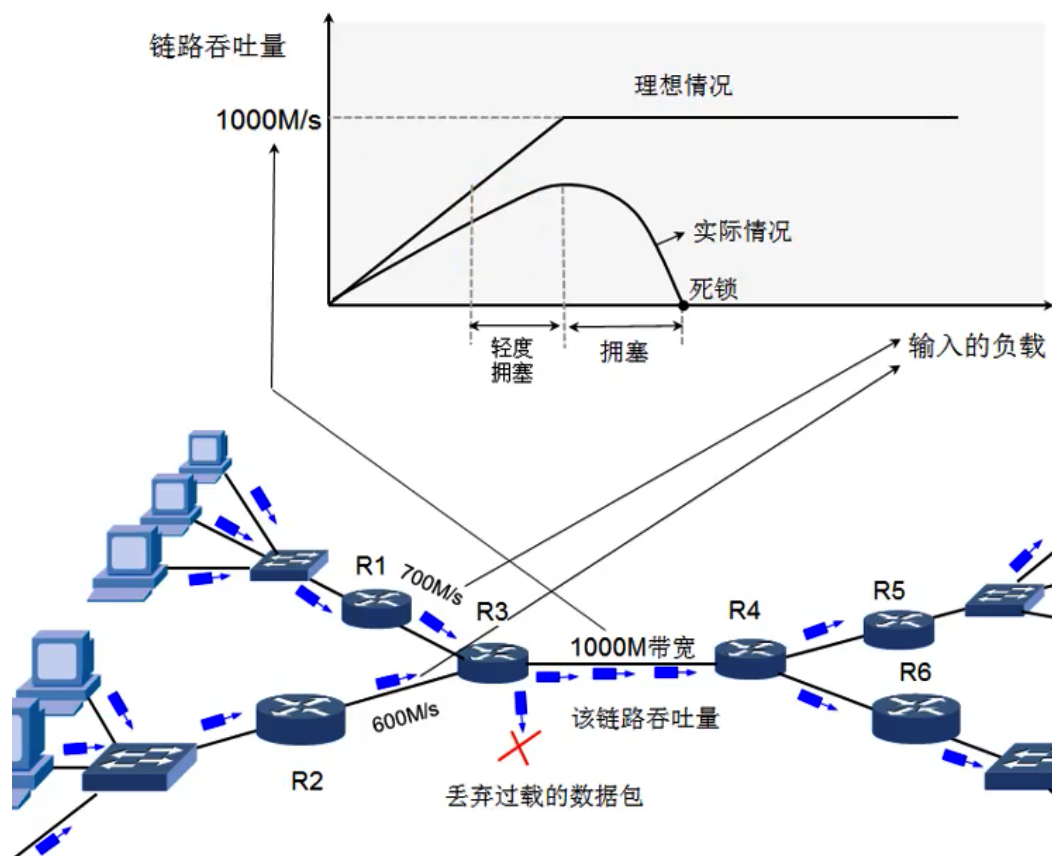
A. 1000 B. 2000 C. 3000 D. 4000

6. TCP拥塞控制

6.1. 什么是拥塞控制

- 拥塞：在某段时间，若对网络中某一资源的需求超过了该资源所能提供的可用部分，网络性能就要变坏，这种情况就叫作拥塞（congestion）。

- 计算机网络中的链路容量（带宽）、交换节点中的缓存和处理机等都是网络的资源。

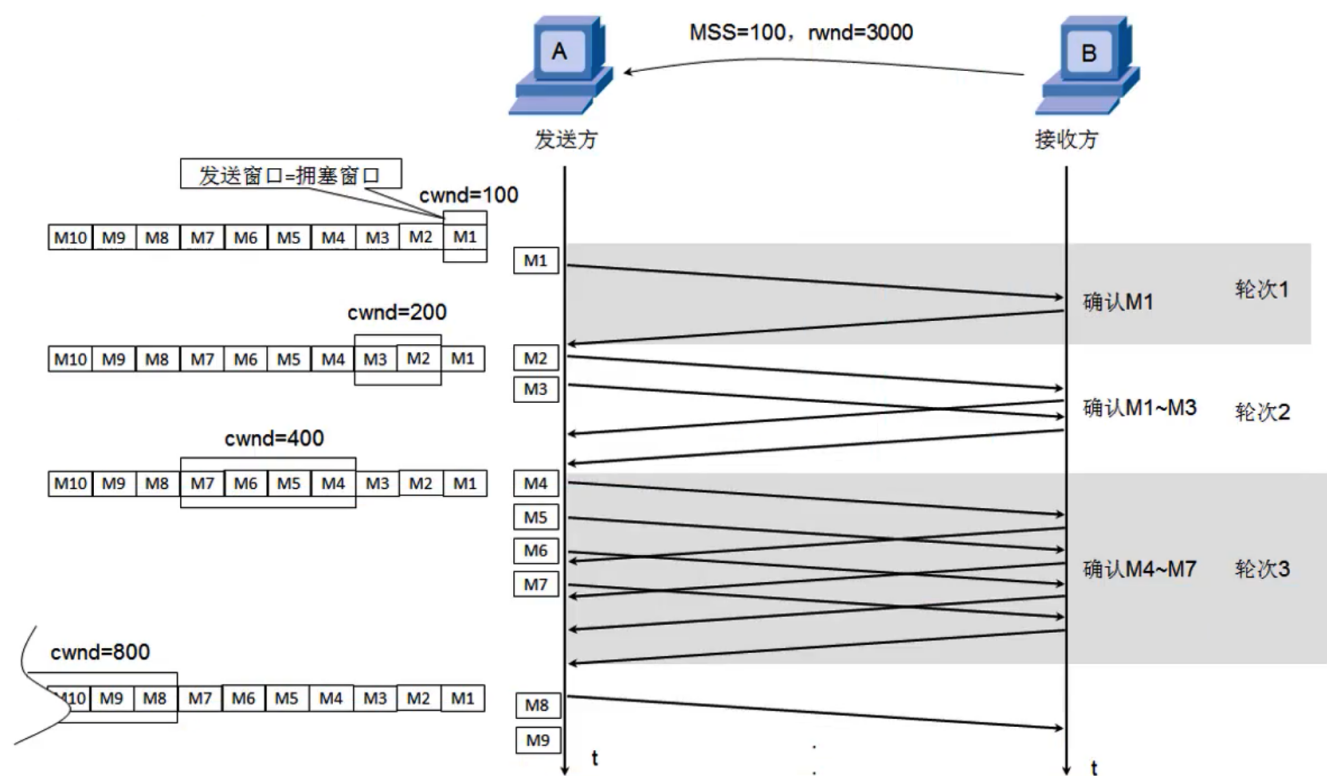


- 拥塞控制是一个全局性的过程
 - 涉及到所有的主机、路由器，以及与降低网络传输性能有关的所有因素
 - 是需要靠所有节点共同努力的结果
 - 防止过多的数据注入到网络中，使网络能够承受现有的网络负荷。
- 相比而言，流量控制是点对点通信的控制

6.2. 拥塞控制的常用算法

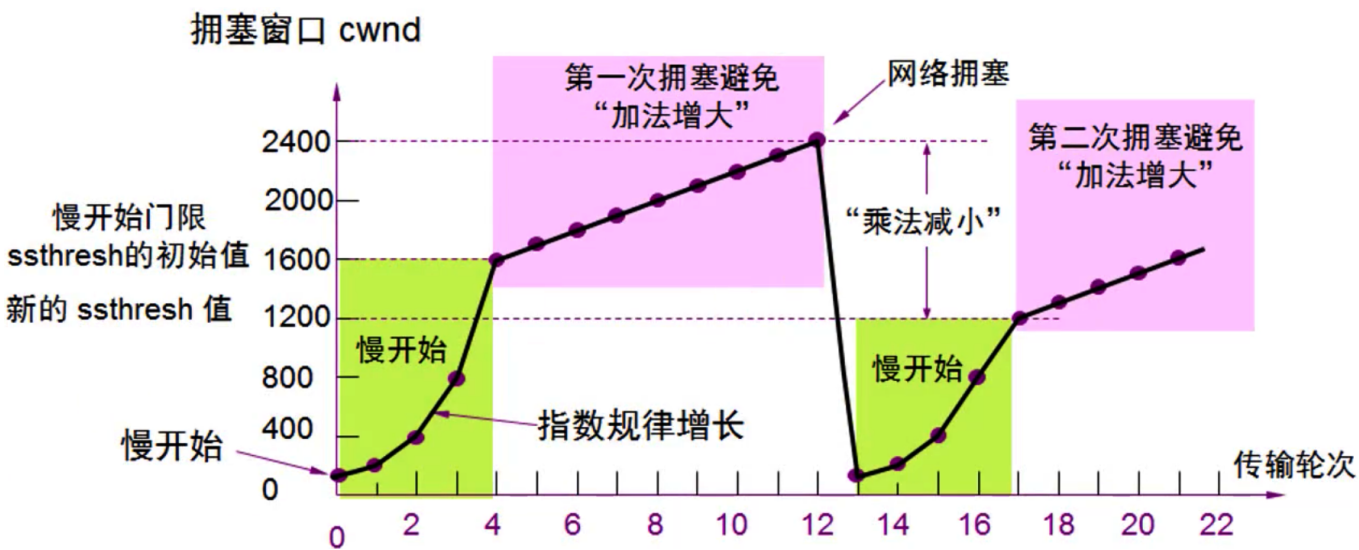
- 方法概述
 - 慢开始 (slow start, 慢启动)、拥塞避免 (congestion avoidance)
 - 快速重传 (fast retransmit)、快速恢复 (fast recovery)
- 几个缩写
 - cwnd (congestion window) : 拥塞窗口
 - rwnd (receive window) : 接收窗口
 - swnd (send window) : 发送窗口, $swnd = \min(cwnd, rwnd)$

6.2.1. 慢开始



- cwnd的初始值比较小，然后随着数据包被接收方确认（收到一个ACK）
 - cwnd成倍增长（指数级）

6.2.2. 拥塞避免



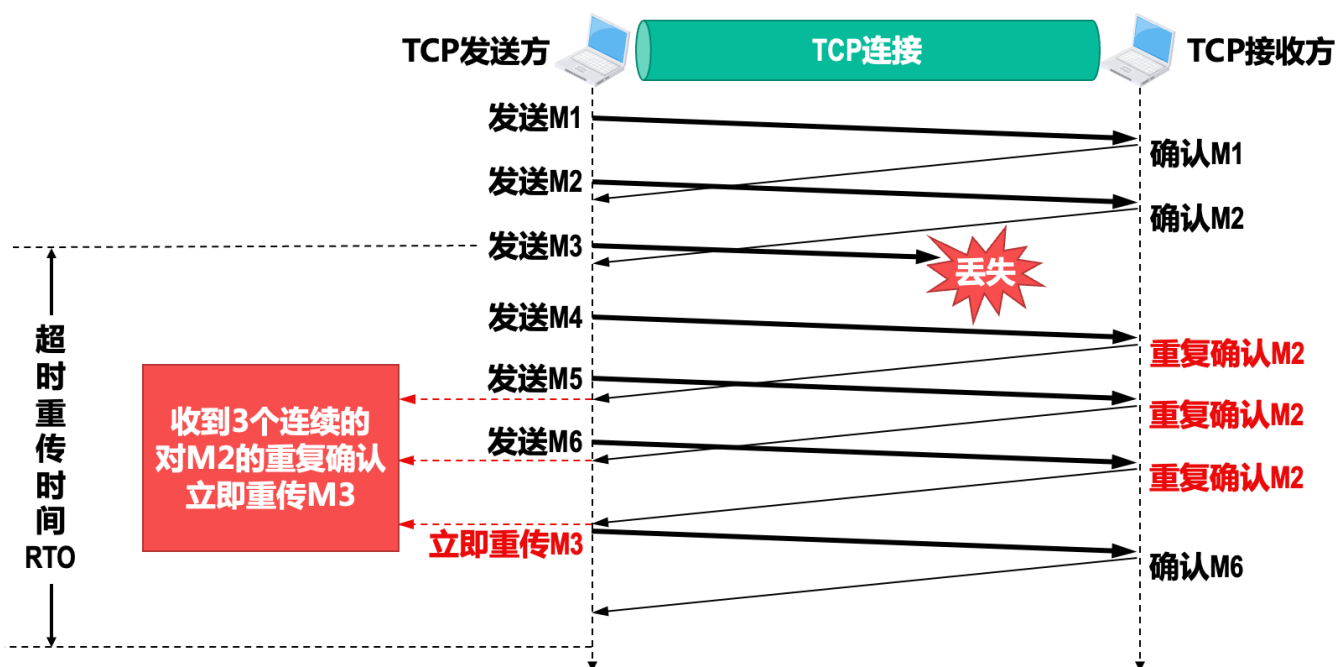
- ssthresh (slow start threshold)：慢开始阈值，cwnd达到阈值后，以线性方式增加
- 拥塞避免（加法增大）：拥塞窗口缓慢增大，以防止网络过早出现拥塞
- 乘法减小：只要网络出现拥塞，把ssthresh减为拥塞峰值的一半，同时执行慢开始算法（cwnd又恢

复到初始值)

- 当网络出现频繁拥塞时，ssthresh值就下降的很快

6.2.3. 快重传

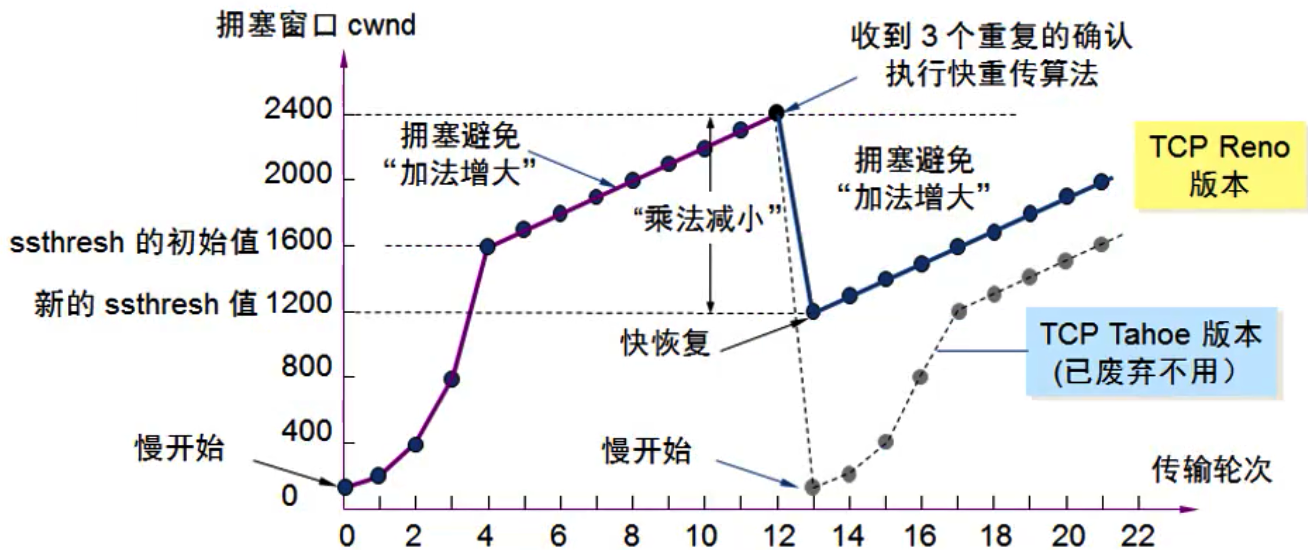
- 接收方
 - 每收到一个失序的分组后就立即发出重复确认
 - 使发送方及时知道有分组没有到达
 - 而不要等待自己发送数据时才进行确认
- 发送方
 - 只要连续收到三个重复确认（总共4个相同的确认），就应当立即重传对方尚未收到的报文段
 - 而不必继续等待重传计时器到期后再重传



对于个别丢失的报文段，发送方不会出现超时重传，也就不会误认为出现了拥塞而错误地把拥塞窗口cwnd的值减为1。实践证明，使用快重传可以使整个网络的吞吐量提高约20%。

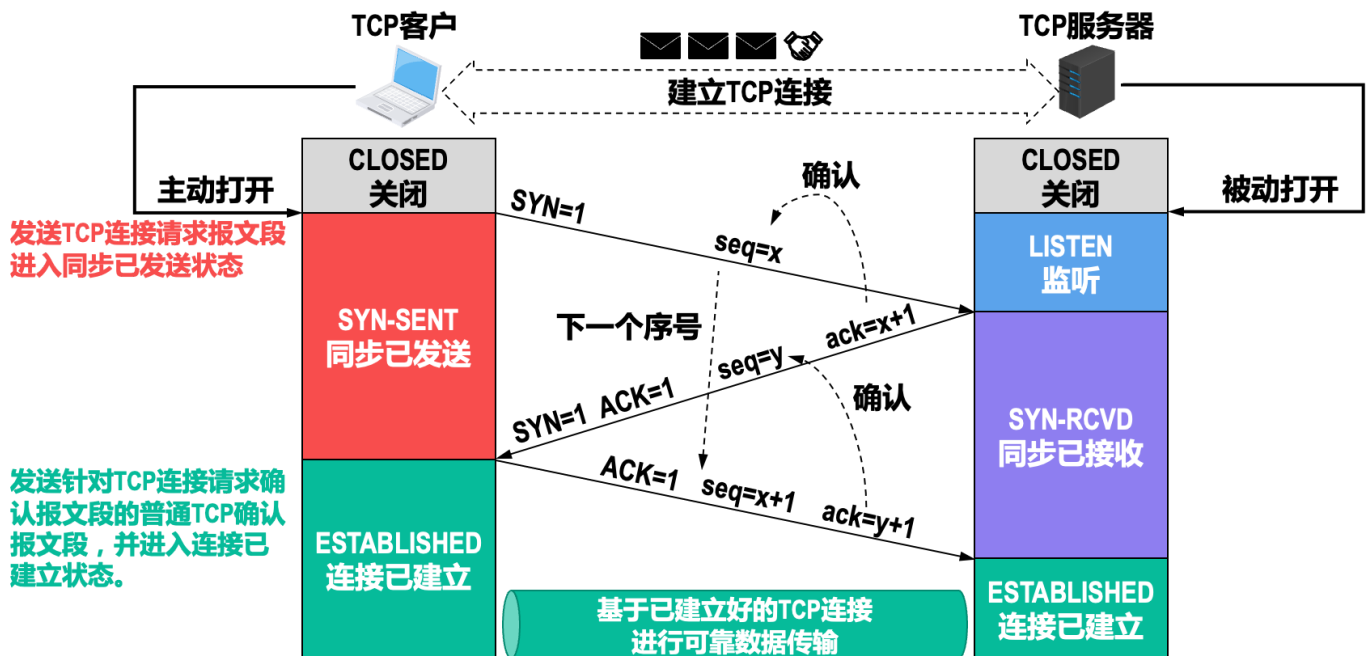
6.2.4. 快恢复

- 当发送方连续收到三个重复确认，说明网络出现拥塞
 - 就执行“乘法减小”算法，把ssthresh减为拥塞峰值的一半
- 与慢开始不同之处是现在不执行慢开始算法，即cwnd现在不恢复到初始值
 - 而是把cwnd值设置为新的ssthresh值（减小后的值）
 - 然后开始执行拥塞避免算法（“加法增大”），使拥塞窗口缓慢地线性增大



7. TCP建立连接和释放连接

7.1. 三次握手



▼ 练习题

1. 【2011年 题39】主机甲向主机乙发送一个 (SYN=1, seq=11220) 的TCP段, 期望与主机乙建立TCP连接, 若主机乙接受该连接请求, 则主机乙向主机甲发送的正确的TCP段可能是 ()。

- A. (SYN=0, ACK=0, seq=11221, ack=11221) B. (SYN=1, ACK=1, seq=11220, ack=11220)
- C. (SYN=1, ACK=1, seq=11221, ack=11221) D. (SYN=0, ACK=0, seq=11220, ack=11220)

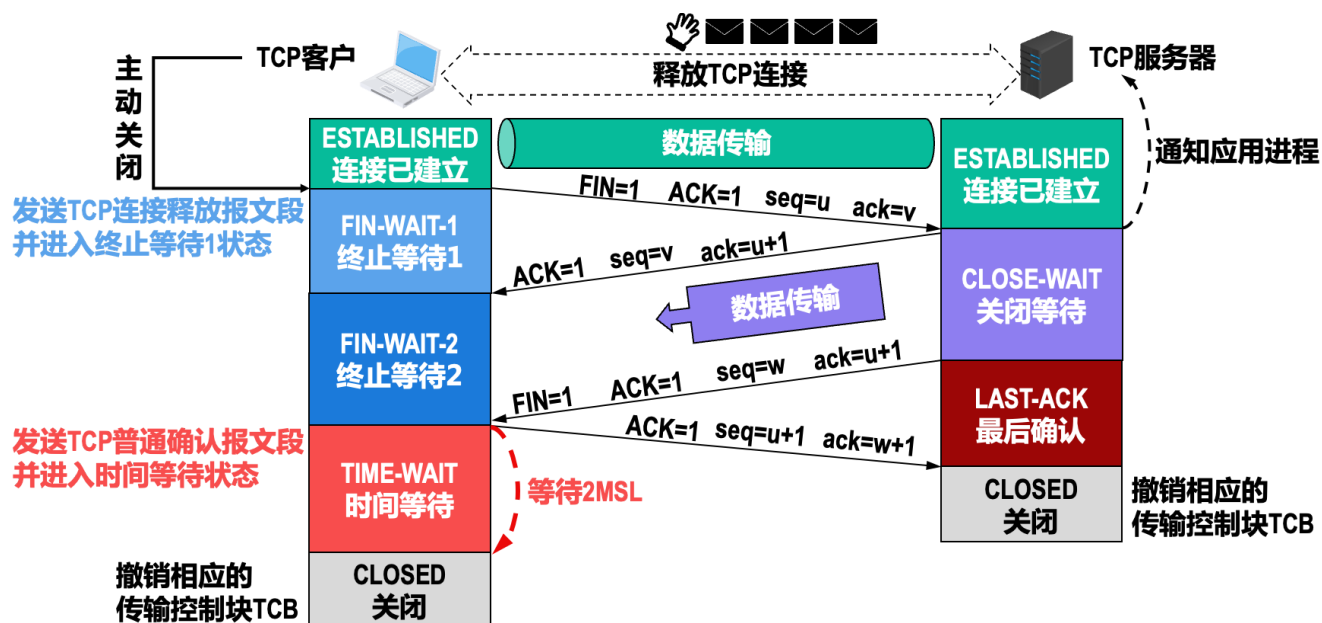
2. 【2019年 题39】若主机甲主动发起一个与主机乙的TCP连接, 甲、乙选择的初始序列号分别为2018和2046, 则第三次握手TCP段的确认序列号是 ()。

- A. 2018 B. 2019 C. 2046 D. 2047

7.1.1. 为什么必须是三次握手而不是二次握手

采用“三报文握手”而不是“两报文握手”来建立TCP连接, 是为了防止已失效的TCP连接请求报文段突然又传送到了TCP服务器进程, 因而导致错误。

7.2. 四次挥手



• 第一次挥手

- 序号seq字段的值设置为u, 它等于TCP客户进程之前已经传送过的数据的最后一个字节的序号加1。
- TCP规定终止标志位FIN等于1的TCP报文段即使不携带数据, 也要消耗掉一个序号。

- 确认号ack字段的值设置为v，它等于TCP客户进程之前已收到的数据的最后一个字节的序号加1。

- 第二次挥手

- 序号seq字段的值设置为v，它等于TCP服务器进程之前已传送过的数据的最后一个字节的序号加1。这也与之前收到的TCP连接释放报文段中的确认号v匹配。
- 确认号ack字段的值设置为u+1，这是对TCP连接释放报文段的确认。

经过两次挥手后，从TCP客户进程到TCP服务器进程这个方向的连接就释放了，但TCP服务器进程如果还有数据要发送，TCP客户进程仍要接收，也就是从TCP服务器进程到TCP客户进程这个方向的连接并未关闭。

- 第三次挥手

- TCP连接释放报文段首部中的终止标志位FIN和确认标志位ACK的值都被设置为1。表明这是一个TCP连接释放报文段，同时也对之前收到的TCP报文段进行确认。
- 序号seq字段的值假定被设置为w，这是因为在半关闭状态下TCP服务器进程可能又发送了一些数据。
- 确认号ack字段的值被设置为u+1，这是对之前收到的TCP连接释放报文段的重复确认。
- TCP服务器进入LAST-ACK状态

- 第四次挥手

- 序号seq字段的值设置为u+1，这是因为TCP客户进程之前发送的TCP连接释放报文段虽然不携带数据，但要消耗掉一个序号。
- 确认号ack字段的值设置为w+1，这是对所收到的TCP连接释放报文段的确认。
- TCP客户端进入TIME-WAIT状态，等待2MLS，MSL是最长报文段寿命（Maximum Segment Lifetime）的英文缩写词，[RFC793]建议为2分钟。也就是说，TCP客户进程进入时间等待（TIME-WAIT）状态后，还要经过4分钟才能进入关闭（CLOSED）状态。

- ▼ 练习题

1. 【2020年 题39】若主机甲与主机乙建立TCP连接时发送的SYN段中的序号为1000，在断开连接时，甲发送给乙的FIN段中的序号为5001，则在无任何重传的情况下，甲向乙已经发送的应用层数据的字节数为（ ）。

A. 4002 B. 4001 C. 4000 D. 3999